Project Report

Subject: Database Management Systems

E-Commerce Backend System

Roll Number	**23BEC153**
Database	**MySQL 8.0**
Language	**Python 3 + Tkinter**
Academic Year	**2024–25**

Contents

List of Tables iii

1 Introduction and Problem Definition 1
- 1.1 Project Overview . . . 1
- 1.2 Objectives . . . 1
- 1.3 Scope . . . 2
- 1.4 Technology Stack . . . 2

2 Entity Relationship Diagram 3
- 2.1 Entities and Attributes . . . 3
- 2.2 Relationships . . . 3
- 2.3 ER Diagram (TikZ) . . . 4

3 Database Schema and Normalization 5
- 3.1 Normalization Analysis . . . 5
- 3.2 Schema Diagram . . . 6

4 SQL Queries — DDL and DML 7
- 4.1 Database Initialisation . . . 7
- 4.2 Table Creation (DDL) . . . 7
- 4.3 Data Insertion (DML) . . . 9
- 4.4 Data Retrieval Queries (DQL) . . . 11

5 Python Tkinter GUI 15
- 5.1 Database Connection Module . . . 15
- 5.2 User Registration Module . . . 15
- 5.3 GUI Modules Summary . . . 17

6 Conclusion 18
- 6.1 Summary of Outcomes . . . 18
- 6.2 Learning Outcomes . . . 19
- 6.3 Future Enhancements . . . 19

References **20**

List of Tables

1.1 Modules and their scope 2
1.2 Technology stack 2

2.1 Entities, primary keys and attributes 3

3.1 Normalization compliance 5

5.1 Tkinter GUI modules and their operations 17

6.1 DBMS concepts learnt and demonstrated 19

Chapter 1

Introduction and Problem Definition

This chapter outlines the motivation, scope and objectives of the E-Commerce Backend Database System developed as part of the **DBMS101 — Database Management Systems** course project.

1.1 Project Overview

The modern e-commerce ecosystem relies on a well-structured backend database to handle high volumes of transactions, product inventories and user accounts efficiently. This project designs and implements a **minimal yet complete** relational database that mirrors the core operations of an online shopping platform.

The system is built on **MySQL 8.0** and exposes a lightweight graphical interface through **Python Tkinter**, making database operations accessible without requiring direct SQL knowledge from the end-user.

1.2 Objectives

1. Model entities — Users, Categories, Products and Orders — using standard **Entity-Relationship (ER) modelling**.
2. Implement a **normalised (up to 3NF)** relational schema in MySQL to eliminate data redundancy.
3. Write **DDL** statements to create all tables with appropriate constraints (PK, FK, UNIQUE, NOT NULL).
4. Write **DML** statements for realistic data insertion and **DQL** statements for mean-

ingful query retrieval.

5. Develop a **Python Tkinter GUI** to interact with the database without manual SQL entry.

1.3 Scope

TableHead	
User Management	Registration, unique email enforcement
TableAlt Category Management	Hierarchical product classification
Product Management	Product listing with price and category
TableAlt Order Processing	Order placement with timestamp tracking
Reporting	JOIN-based cross-table query reports
TableAlt GUI Interface	Tkinter forms for CRUD operations

Table 1.1: Modules and their scope

1.4 Technology Stack

TableHead		
RDBMS	MySQL	8.0
TableAlt Backend Language	Python	3.11
GUI Framework	Tkinter	Built-in
TableAlt DB Connector	mysql-connector-python	8.x
OS	Windows / Linux	—

Table 1.2: Technology stack

Chapter 2

Entity Relationship Diagram

The ER diagram captures all entities, their attributes and the relationships among them before translating them into relational tables.

2.1 Entities and Attributes

TableHead		
Users	user_id	name, email (UNIQUE), password
TableAlt Categories	category_id	category_name
Products	product_id	product_name, price, category_id (FK)
TableAlt Orders	order_id	user_id (FK), order_date

Table 2.1: Entities, primary keys and attributes

2.2 Relationships

- **Users → Orders** (One-to-Many) — A single user can place multiple orders.
- **Categories → Products** (One-to-Many) — A category groups multiple products.
- **Orders → Products** (Many-to-Many via Order_Items) — An order can contain multiple products; a product can appear in multiple orders.

2.3 ER Diagram (TikZ)

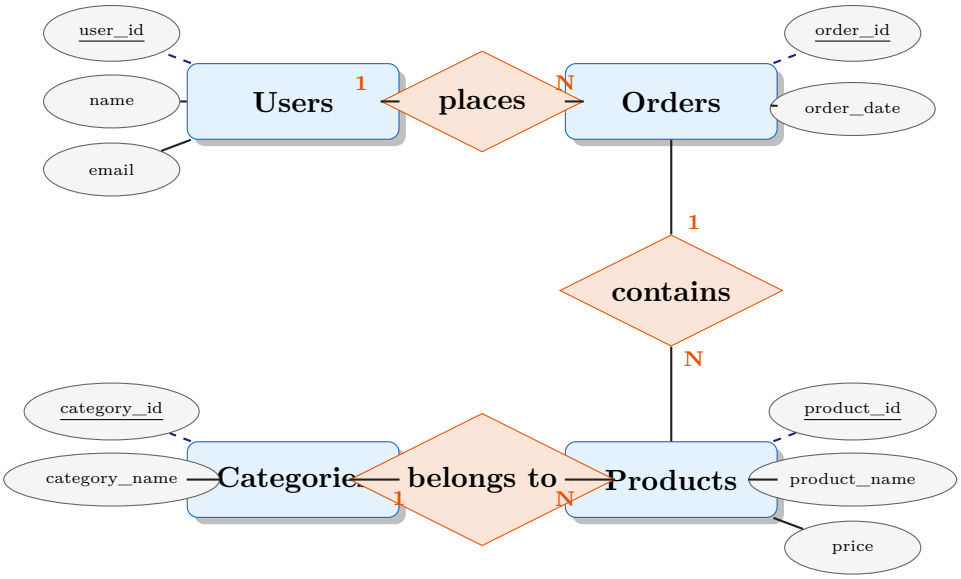

Figure 2.1: Entity Relationship Diagram — E-Commerce Backend System

Note

Attributes with an underline represent **Primary Keys**. Dashed blue lines connect an entity to its primary-key attribute. Cardinality notations (1 and N) are placed adjacent to the relationship diamond.

Chapter 3

Database Schema and Normalization

3.1 Normalization Analysis

TableHead	
1NF	All columns contain atomic values; no repeating groups.
TableAlt 2NF	Every non-key attribute is fully functionally dependent on the whole primary key (all PKs are single-column surrogate keys).
3NF	No transitive dependencies exist; `category_name` is stored in `Categories`, not repeated in `Products`.

Table 3.1: Normalization compliance

3.2 Schema Diagram

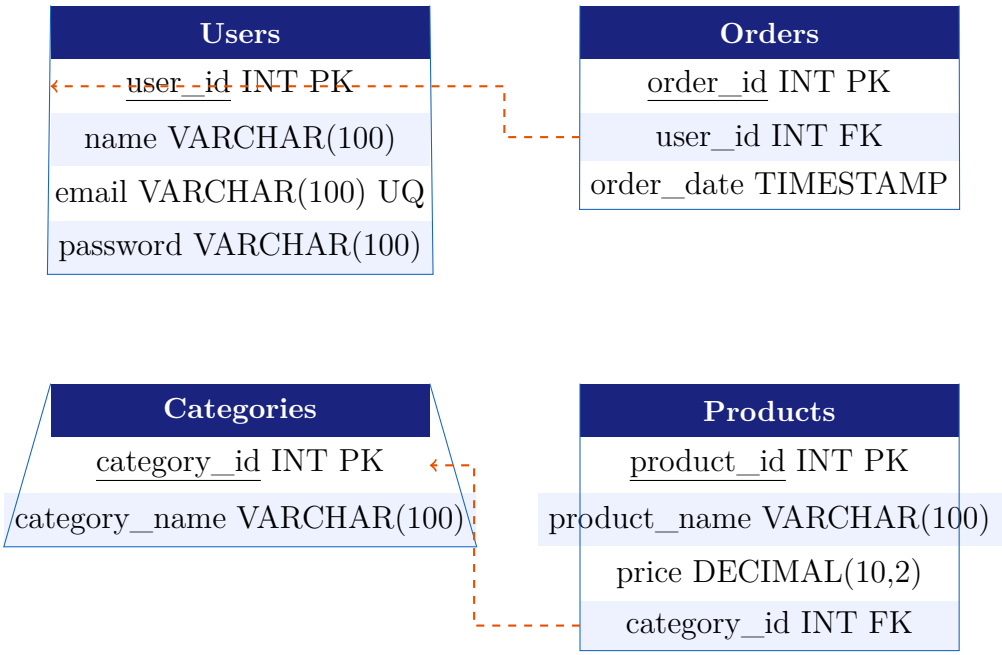

Figure 3.1: Relational Schema Diagram with Foreign Key references

Chapter 4

SQL Queries — DDL and DML

All queries are categorised using badges: DDL Data Definition Language DML Data Manipulation Language DQL Data Query Language.

4.1 Database Initialisation

Query 1 — Create and Select Database DDL

```sql
CREATE DATABASE IF NOT EXISTS ecommerce_db
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

USE ecommerce_db;
```

Listing 4.1: Create and use the e-commerce database

Query Output

Database `ecommerce_db` created and selected successfully.

4.2 Table Creation (DDL)

Query 2 — Create `Users` Table DDL

```sql
CREATE TABLE IF NOT EXISTS Users (
    user_id    INT            AUTO_INCREMENT PRIMARY KEY,
    name       VARCHAR(100)   NOT NULL,
    email      VARCHAR(100)   NOT NULL UNIQUE,
    password   VARCHAR(255)   NOT NULL,
```

```
    created_at TIMESTAMP      DEFAULT CURRENT_TIMESTAMP
);
```

Listing 4.2: DDL — Users table

Query Output

Table `Users` created successfully.

Constraints enforced: `PRIMARY KEY`, `UNIQUE(email)`, `NOT NULL`.

Query 3 — Create Categories Table DDL

```sql
CREATE TABLE IF NOT EXISTS Categories (
    category_id   INT           AUTO_INCREMENT PRIMARY KEY,
    category_name VARCHAR(100) NOT NULL UNIQUE
);
```

Listing 4.3: DDL — Categories table

Query Output

Table `Categories` created successfully.

Query 4 — Create Products Table DDL

```sql
CREATE TABLE IF NOT EXISTS Products (
    product_id   INT             AUTO_INCREMENT PRIMARY KEY,
    product_name VARCHAR(100)    NOT NULL,
    price        DECIMAL(10,2)   NOT NULL CHECK (price >= 0),
    stock        INT             NOT NULL DEFAULT 0,
    category_id  INT,
    CONSTRAINT fk_product_category
        FOREIGN KEY (category_id)
        REFERENCES Categories(category_id)
        ON DELETE SET NULL
        ON UPDATE CASCADE
);
```

Listing 4.4: DDL — Products table with FK to Categories

Query Output

Table `Products` created successfully.
Foreign key `fk_product_category` references `Categories(category_id)`.

Query 5 — Create Orders Table DDL

```sql
CREATE TABLE IF NOT EXISTS Orders (
    order_id   INT       AUTO_INCREMENT PRIMARY KEY,
    user_id    INT       NOT NULL,
    order_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    status
       ENUM('pending','confirmed','shipped','delivered','cancelled')
               DEFAULT 'pending',
    CONSTRAINT fk_order_user
        FOREIGN KEY (user_id)
        REFERENCES Users(user_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);
```

Listing 4.5: DDL — Orders table with FK to Users

Query Output

Table `Orders` created successfully.
Status ENUM restricts values to valid order states.

4.3 Data Insertion (DML)

Query 6 — Insert Users DML

```sql
INSERT INTO Users (name, email, password)
VALUES
    ('Rahul Sharma', 'rahul@gmail.com',  SHA2('Pass@1234', 256)),
    ('Amit Patel',   'amit@gmail.com',   SHA2('Secure#99', 256)),
    ('Priya Mehta',  'priya@gmail.com',  SHA2('MyPass!55', 256));
```

Listing 4.6: DML — Inserting user records

Query Output

3 rows inserted into `Users`.
Passwords hashed using `SHA2` (256-bit) for security.

Query 7 — Insert Categories DML

```sql
INSERT INTO Categories (category_name)
VALUES
    ('Electronics'),
    ('Clothing'),
    ('Books'),
    ('Home Appliances'),
    ('Sports');
```

Listing 4.7: DML — Inserting product categories

Query Output

5 rows inserted into `Categories`.

Query 8 — Insert Products DML

```sql
INSERT INTO Products (product_name, price, stock,
   category_id)
VALUES
    ('Laptop 15"',          65000.00,  30, 1),
    ('Wireless Headphones', 3500.00, 50, 1),
    ('Cotton T-Shirt',      1200.00, 200, 2),
    ('Denim Jeans',         2500.00, 150, 2),
    ('Database Systems',     800.00,  75, 3),
    ('Clean Code',           650.00,  60, 3),
    ('Air Conditioner',    35000.00,  15, 4),
    ('Cricket Bat',          1800.00,  40, 5);
```

Listing 4.8: DML — Inserting products with categories

Query Output

8 rows inserted into `Products`.

Query 9 — Create Orders DML

```sql
INSERT INTO Orders (user_id, status)
VALUES
    (1, 'confirmed'),
    (2, 'pending'),
    (1, 'delivered'),
    (3, 'shipped');
```

Listing 4.9: DML — Placing orders for users

Query Output

4 rows inserted into `Orders`.

4.4 Data Retrieval Queries (DQL)

Query 10 — List All Users DQL

```sql
SELECT user_id, name, email, created_at
FROM   Users
ORDER BY user_id;
```

Listing 4.10: DQL — Retrieve all registered users

Query Output

user_id	name	email	created_at
1	Rahul Sharma	rahul@gmail.com	2025-01-10 09:00:00
TableAlt 2	Amit Patel	amit@gmail.com	2025-01-11 10:30:00
3	Priya Mehta	priya@gmail.com	2025-01-12 11:00:00

Query 11 — JOIN: User Orders DQL

```sql
SELECT
    U.user_id,
```

```sql
    U.name           AS customer_name,
    O.order_id,
    O.order_date,
    O.status
FROM  Users  U
JOIN  Orders O ON U.user_id = O.user_id
ORDER BY O.order_date DESC;
```

Listing 4.11: DQL — Inner join between Users and Orders

Query Output

user_id	customer_name	order_id	order_date	status
3	Priya Mehta	4	2025-01-14	shipped
TableAlt 1	Rahul Sharma	3	2025-01-13	delivered
2	Amit Patel	2	2025-01-12	pending
TableAlt 1	Rahul Sharma	1	2025-01-11	confirmed

Query 12 — JOIN: Products with Categories DQL

```sql
SELECT
    P.product_id,
    P.product_name,
    P.price,
    P.stock,
    C.category_name
FROM  Products   P
JOIN  Categories C ON P.category_id = C.category_id
ORDER BY C.category_name, P.price DESC;
```

Listing 4.12: DQL — Products joined with category names

Query Output

ID	Product	Category	Price ()	Stock
7	Air Conditioner	Home Appliances	35000	15
TableAlt 1	Laptop 15"	Electronics	65000	30
2	Wireless Headphones	Electronics	3500	50
TableAlt 4	Denim Jeans	Clothing	2500	150
3	Cotton T-Shirt	Clothing	1200	200
TableAlt 5	Database Systems	Books	800	75
6	Clean Code	Books	650	60
TableAlt 8	Cricket Bat	Sports	1800	40

Query 13 — Aggregate: Orders per User DQL

```sql
SELECT
    U.name              AS customer,
    COUNT(O.order_id)   AS total_orders
FROM  Users  U
LEFT JOIN Orders O ON U.user_id = O.user_id
GROUP BY U.user_id, U.name
ORDER BY total_orders DESC;
```

Listing 4.13: DQL — Count orders per user using GROUP BY

Query Output

Customer	Total Orders
Rahul Sharma	2
TableAlt Amit Patel	1
Priya Mehta	1

Query 14 — Products above Average Price DQL

```sql
SELECT product_name, price
FROM   Products
WHERE  price > (SELECT AVG(price) FROM Products)
ORDER BY price DESC;
```

Listing 4.14: DQL — Subquery to find above-average priced products

Query Output

Returns products priced above the average of 13,931.25:

Product Name	Price ()
Laptop 15"	65000
TableAlt Air Conditioner	35000

Query 15 — UPDATE Order Status DML

```sql
UPDATE Orders
SET    status = 'delivered'
WHERE  order_id = 2;
```

Listing 4.15: DML — Update order status to delivered

Query Output

1 row affected. `Orders.status` updated to `'delivered'` for `order_id = 2`.

Query 16 — DELETE a User DML

```sql
DELETE FROM Users
WHERE user_id = 3;
-- Cascades to Orders: all orders by user_id=3 are deleted
```

Listing 4.16: DML — Delete a user (CASCADE removes their orders)

Query Output

1 row deleted from `Users`.

Due to `ON DELETE CASCADE`, related rows in `Orders` are also removed.

Chapter 5

Python Tkinter GUI

A lightweight graphical interface is developed using Python's built-in `Tkinter` library and the `mysql-connector-python` package.

5.1 Database Connection Module

```python
import mysql.connector

def get_connection():
    """Return a live MySQL connection."""
    return mysql.connector.connect(
        host     = "localhost",
        user     = "root",
        password = "your_password",
        database = "ecommerce_db"
    )
```

Listing 5.1: Python — MySQL connection helper

5.2 User Registration Module

```python
import tkinter as tk
from tkinter import messagebox

def register_user(name, email, password):
    conn   = get_connection()
    cursor = conn.cursor()
    try:
```

```python
        cursor.execute(
            "INSERT INTO Users (name, email, password) "
            "VALUES (%s, %s, SHA2(%s, 256))",
            (name, email, password)
        )
        conn.commit()
        messagebox.showinfo("Success", "User registered successfully!")
    except mysql.connector.IntegrityError:
        messagebox.showerror("Error", "Email already exists.")
    finally:
        cursor.close()
        conn.close()

#         Build the form window
root = tk.Tk()
root.title("User Registration")
root.geometry("400x300")
root.configure(bg="#F5F5F5")

tk.Label(root, text="Name",     bg="#F5F5F5").pack(pady=4)
name_entry = tk.Entry(root, width=30); name_entry.pack()

tk.Label(root, text="Email",    bg="#F5F5F5").pack(pady=4)
email_entry = tk.Entry(root, width=30); email_entry.pack()

tk.Label(root, text="Password", bg="#F5F5F5").pack(pady=4)
pass_entry  = tk.Entry(root, width=30, show="*"); pass_entry.pack()

tk.Button(
    root, text="Register",
    command=lambda: register_user(
        name_entry.get(), email_entry.get(), pass_entry.get()),
    bg="#1565C0", fg="white", padx=10
).pack(pady=14)

root.mainloop()
```

Listing 5.2: Python — Register a new user via GUI

5.3 GUI Modules Summary

TableHead	
User Registration	INSERT new user; duplicate email detection
TableAlt Product Management	INSERT / SELECT products; category dropdown
Order Management	INSERT order; display order history per user
TableAlt Reports Window	JOIN query results in a scrollable table

Table 5.1: Tkinter GUI modules and their operations

Chapter 6

Conclusion

This chapter summarises the outcomes of the project and reflects on the DBMS concepts demonstrated.

6.1 Summary of Outcomes

The E-Commerce Backend Database System successfully demonstrates:

- **ER Modelling** — all entities, attributes and relationships are accurately captured in the ER diagram.
- **Relational Schema** — the schema is normalised up to **3NF**, eliminating both partial and transitive dependencies.
- **DDL Mastery** — all tables are created with proper `PRIMARY KEY`, `FOREIGN KEY`, `UNIQUE`, `NOT NULL` and `CHECK` constraints.
- **DML and DQL** — realistic data was inserted and retrieved using `JOIN`, `GROUP BY`, `ORDER BY`, subqueries and aggregate functions.
- **Cascade Operations** — `ON DELETE CASCADE` and `ON UPDATE CASCADE` ensure referential integrity automatically.
- **Python Integration** — the `mysql-connector-python` package bridges the application layer to MySQL, and the Tkinter GUI makes CRUD operations accessible to non-technical users.

6.2 Learning Outcomes

TableHead	
ER Modelling	4-entity diagram with cardinalities
TableAlt Normalisation	1NF → 3NF analysis
SQL DDL	`CREATE TABLE` with all constraint types
TableAlt SQL DML	`INSERT`, `UPDATE`, `DELETE`
SQL DQL	`SELECT` with `JOIN`, aggregates, subqueries
TableAlt Python-DB Bridge	`mysql-connector-python` parameterised queries
GUI Development	Tkinter forms, event handling, error dialogs

Table 6.1: DBMS concepts learnt and demonstrated

6.3 Future Enhancements

1. Add an `Order_Items` junction table for a true many-to-many Product–Order relationship.
2. Implement a **shopping cart** entity with session management.
3. Apply **stored procedures** for complex multi-table transaction operations.
4. Integrate **views** for frequently accessed report queries.
5. Migrate the GUI to a **web interface** using Flask or Django.

References

[1] Ramakrishnan, R., & Gehrke, J. (2003). *Database Management Systems* (3rd ed.). McGraw-Hill.

[2] Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database System Concepts* (7th ed.). McGraw-Hill.

[3] MySQL 8.0 Reference Manual. Oracle Corporation. https://dev.mysql.com/doc/refman/8.0/en/

[4] Python Software Foundation. (2024). *Tkinter — Python interface to Tcl/Tk*. https://docs.python.org/3/library/tkinter.html

[5] mysql-connector-python Documentation. https://dev.mysql.com/doc/connector-python/en/